

Week 2: Python Basics – Student Grade Calculator

1. Project Overview

This project was developed as part of Week 2 of my Data Science Internship at The Developers Arena.

The Student Grade Calculator is a beginner-friendly Python program that accepts a student's name and marks, validates the marks, determines the appropriate grade, and displays an encouraging message.

The project demonstrates decision-making and repetition in Python through if-elif-else statements, comparison logic, a while loop for input validation, functions, user input, and formatted output.

2. Objectives

The main objectives of this project are:

- To understand if-elif-else statements for decision-making.
- To use comparison operators for grading conditions.
- To understand and use while loops.
- To create and use Python functions.
- To validate user input.
- To handle invalid and non-numeric input.
- To calculate grades according to the specified grading ranges.
- To display meaningful and encouraging messages.

3. Technologies Used

- Python 3.x
- Visual Studio Code
- Git
- GitHub

No external Python libraries are required for this project.

4. Features

- Student name input
- Marks input

- Marks validation from 0 to 100
- Grade calculation
- If-elif-else grading logic
- While loop for invalid input
- Function-based grade calculation
- Encouraging messages for each grade
- Handling of non-numeric input
- Simple and beginner-friendly console interface

5. Grading Logic

The program uses the following grading system:

Marks Grade

90–100 A

80–89 B

70–79 C

60–69 D

0–59 F

6. Project Structure

The Week 2 project is organized in a separate folder inside the internship repository.

Task_2_Student_Grade_Calculator/

|

|— grade_calculator.py

|— README.md

|— test_cases.txt

|— screenshots/

File Description

grade_calculator.py

Contains the main Python program for calculating the student's grade.

README.md

Contains project overview, setup instructions, features, technical details, and usage information.

test_cases.txt

Contains different test cases used to verify the grading logic and input validation.

screenshots/

Contains screenshots showing the working application and testing evidence.

7. Setup Instructions

Python 3.x is required to run this project.

To check whether Python is installed, open the VS Code terminal or PowerShell and run:

```
python --version
```

Then open the project folder in Visual Studio Code.

Navigate to:

```
Task_2_Student_Grade_Calculator
```

8. How to Run the Program

Open the terminal inside the Task_2_Student_Grade_Calculator folder.

Run:

```
python grade_calculator.py
```

The program will ask for the student's name and marks.

After entering valid marks, the program calculates and displays the appropriate grade and an encouraging message.

9. Code Structure

The program follows a simple step-by-step structure:

1. Ask the user to enter the student's name.
2. Ask the user to enter marks.
3. Validate the entered marks.
4. Use a while loop to repeatedly ask for marks if the input is invalid.
5. Use a function to calculate the grade.

6. Use if-elif-else statements to determine the grade.
7. Display the student's name, marks, grade, and encouraging message.

10. Technical Requirements Fulfilled

If-elif-else Statements

The program uses if-elif-else statements to determine whether the student's marks correspond to grade A, B, C, D, or F.

Input Validation

The program accepts marks only between 0 and 100.

Invalid values such as negative numbers or numbers greater than 100 are rejected.

Function

At least one function is used to keep the grade calculation organized and reusable.

Encouraging Messages

The program displays a positive and encouraging message according to the student's grade.

While Loop

A while loop is used to repeatedly request marks until the user enters valid input.

User Input

The input() function is used to collect the student's name and marks.

11. Example Input and Output

Example input:

Enter student name: Priya

Enter marks (0-100): 85

Example output:

RESULT FOR PRIYA:

Marks: 85/100

Grade: B

Message: Very Good! Keep working hard! 👍

12. Testing

The following test cases are used to verify the program:

Test Case	Input	Expected Result
-----------	-------	-----------------

A Grade	95	A
---------	----	---

B Grade	85	B
---------	----	---

C Grade	75	C
---------	----	---

D Grade	65	D
---------	----	---

F Grade	50	F
---------	----	---

Below Range -10	Reject and ask again	
-----------------	----------------------	--

Above Range 110	Reject and ask again	
-----------------	----------------------	--

Non-numeric abc	Reject and ask again	
-----------------	----------------------	--

The program should correctly calculate all five grades and handle invalid input without terminating unexpectedly.

13. Visual Documentation

The screenshot shows a Visual Studio Code editor with a dark theme. The Explorer sidebar on the left shows a project structure with folders 'Task_1_Python_Basics' and 'Task_2_Student_Grade_Calculator'. The file 'grade_calculator.py' is selected. The main editor area shows the code for 'grade_calculator.py', which includes a docstring and a function 'calculate_grade'. The bottom panel shows the 'Terminal' output, which displays the program's execution: it prompts for a student name and marks, calculates the grade (B for 80 marks), and prints an encouraging message.

```
1 """Student Grade Calculator for Week 2 of The Developers Arena internship."""
2
3
4 def calculate_grade(marks: int) -> tuple[str, str]:
5     """Return the grade and an encouraging message for the given marks."""
6     # The conditions are checked from the highest grade to the lowest grade.
```

```
D:\Developer_Arena\Week1_Task>cd Task_2_Student_Grade_Calculator
D:\Developer_Arena\Week1_Task\Task_2_Student_Grade_Calculator>python grade_calculator.py
=====
STUDENT GRADE CALCULATOR
=====

Enter student name: Siddhesh
Enter marks (0-100): 80

Student: Siddhesh
Marks: 80/100
Grade: B
Message: Very good! Keep working hard!

D:\Developer_Arena\Week1_Task\Task_2_Student_Grade_Calculator>
```

14. Testing Evidence

The program should be executed using:

```
python grade_calculator.py
```

Testing should verify:

- A grade calculation.
- B grade calculation.
- C grade calculation.
- D grade calculation.
- F grade calculation.
- Marks below 0.
- Marks above 100.
- Non-numeric input.

Testing Screenshot:

[Insert actual terminal screenshot here]

15. What I Learned

Through this project, I learned:

- How to use conditional statements for decision-making.
- How comparison operators work in grading conditions.
- How to use while loops for repeated input.
- How to create and use functions in Python.
- How to validate user input.
- How to handle invalid input.
- How to organize a Python project.
- How to test a Python program.
- How to maintain project files using Git and GitHub.

16. Conclusion

The Student Grade Calculator provided practical experience with Python decision-making, loops, functions, input validation, and structured program design.

The project follows the Week 2 internship requirements while keeping the implementation simple, clean, and beginner-friendly.